

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Source code: https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_7_autoencoders

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

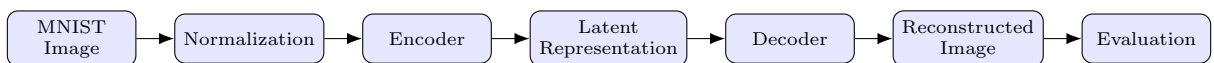
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

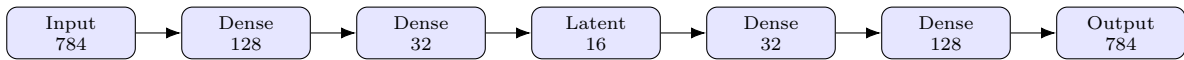
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

Experimental Procedure and Results

Data preparation. The MNIST dataset was loaded through Keras, which provides 60,000 training and 10,000 test images. In line with the recommended laboratory subset, only the first 10,000 training images and the first 2,000 test images were used. Pixel intensities were scaled from $[0, 255]$ to $[0, 1]$, and the minimum and maximum values of the scaled training set were confirmed to be 0.0 and 1.0. The fully connected model takes flattened 784-dimensional vectors, whereas the convolutional, denoising and variational models take the $28 \times 28 \times 1$ images.

Array	Training	Test
Raw images	(10000, 28, 28)	(2000, 28, 28)
Flattened (FC-AE)	(10000, 784)	(2000, 784)
With channel axis (CAE, DAE, VAE)	(10000, 28, 28, 1)	(2000, 28, 28, 1)

Model and training. The fully connected autoencoder follows the architecture given above (784–128–32–16–32–128–784) with ReLU in the hidden layers and a sigmoid output. It was trained with the configuration suggested in the manual (Adam, learning rate 10^{-3} , batch size 128, 20 epochs, binary cross-entropy), using the input image itself as the target. The 2,000 test images were supplied as validation data so that the loss could be monitored during training.

Block	Output shape	Parameters
Encoder (784→128→32→16)	(None, 16)	105,136
Decoder (16→32→128→784)	(None, 784)	105,904
Total		211,040

During training the loss decreased from 0.3461 (training) and 0.2424 (validation) in the first epoch to 0.1263 and 0.1289 in the twentieth epoch.

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

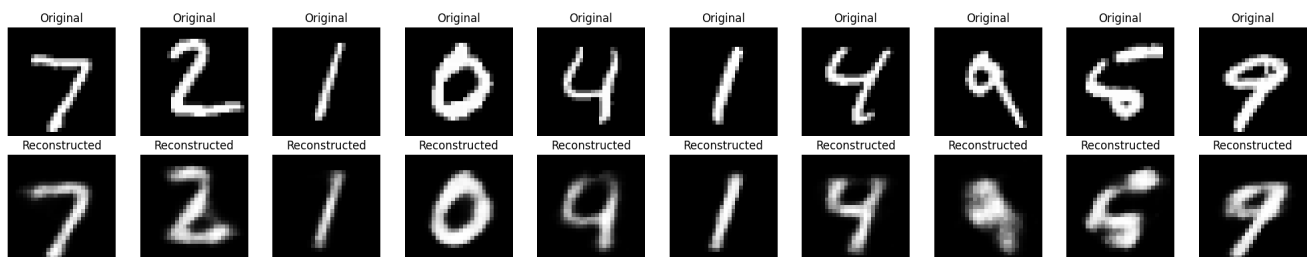
- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

Plot 2: Training and Validation Reconstruction Loss

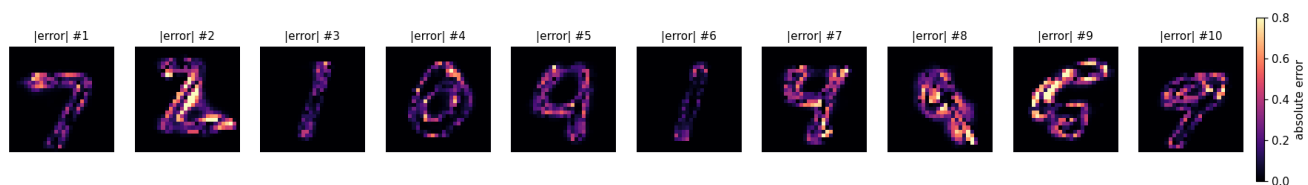
- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

Required inference: Determine whether the model converges and whether there is evidence of overfitting.

Results: Plots 1 and 2



Plot 1: Original test images (top row) and their FC-AE reconstructions (bottom row).



Plot 1 (continued): Absolute reconstruction error $|x - \hat{x}|$ for the ten images of Plot 1. Brighter pixels indicate larger errors.

Comments on Plot 1.

- **Preservation of digit shape.** The overall shape is retained for most images. The 7, the 2, both 1s and the 0 remain easy to recognise and lie close to their originals.
- **Loss of fine details.** Thin features are smoothed out. The thin top bar of the 5 and the narrow gaps inside the loops of the 4s are only partly reproduced.
- **Blurred regions.** The error maps show that the errors are concentrated along the stroke boundaries, especially in curved parts and loops. The 9 in the eighth column, which has a long tail, collapses into an unstructured blob, and the 4 in the fifth column acquires an almost closed top so that it resembles a 9.
- **Easy versus difficult samples.** Thick and nearly straight glyphs such as the 1s, the 0 and the 7 are the easiest, and the two 1s have the lowest error of the ten (about 0.003). Glyphs with thin curved strokes, open tops or tails (the 4s, the 5 and the 9s) are harder, and the 5 has the highest error of the ten (about 0.05).

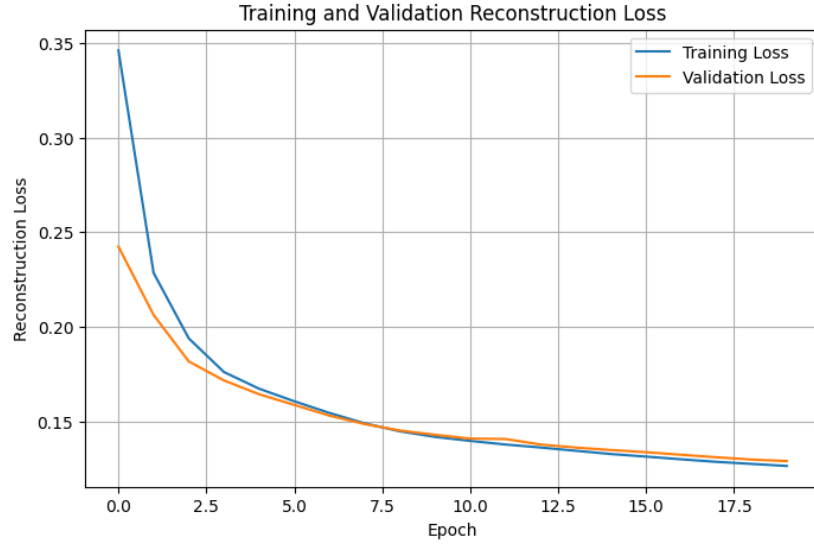
Inference

What the plot shows. The first ten test digits, their reconstructions by the FC-AE with a 16-dimensional code, and the pixel-wise absolute error.

Trend observed. The reconstructions keep the coarse shape of the digit but are blurred. The error is close to zero over the background and is largest along stroke edges, and it varies strongly between images, from about 0.003 for a simple 1 to about 0.05 for the 5.

Why it may occur. A code of 16 numbers cannot store the exact position of every stroke edge, so the decoder produces a smoothed average of similar digits. Simple glyphs need very little information to describe, whereas complex glyphs need much more.

Conclusion. The FC-AE captures the global structure of the digits but not their fine detail. Reconstruction difficulty depends on the complexity of the glyph.



Plot 2: Training and validation reconstruction loss (binary cross-entropy) of the FC-AE over 20 epochs.

Inference

What the plot shows. The training and validation reconstruction loss of the FC-AE for each of the 20 epochs.

Trend observed. Both curves fall quickly during the first four epochs (training 0.346 → 0.176, validation 0.242 → 0.172) and then decrease slowly to 0.1263 and 0.1289 at epoch 20. The validation loss is lower than the training loss up to about epoch 8, the two curves cross around epochs 8–9, and from then on the validation loss stays marginally above the training loss (final difference about 0.003).

Why it may occur. Keras reports the training loss as an average over the epoch, during which the weights are still improving, so it appears higher than the validation loss early on. The small gap at the end is the usual generalisation gap. The binary cross-entropy also has a non-zero floor for grey-valued pixels, so its absolute value cannot be compared with the MSE.

Conclusion. The model converges smoothly. There is no evidence of overfitting, because the validation loss never increases and stays within about 2% of the training loss. The curves are still falling slightly at epoch 20, so a longer run could give a small additional improvement.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

Report:

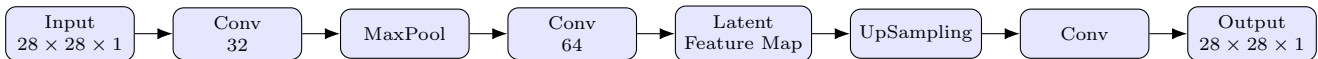
Metric	Value
Test MSE	0.022668
Test MAE	0.059465
Mean SSIM	0.728743

On the 2,000 test images the FC-AE achieves a mean squared error of about 0.023 and a mean SSIM of 0.729. The coarse layout of the digits is therefore preserved, but the local structure is only moderately matched, which agrees with the blur seen in Plot 1.

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

Convolutional Autoencoder: Implementation and Results

The convolutional autoencoder uses the layer sequence listed above, with UpSampling2D layers in the decoder. It was compiled with Adam (learning rate 10^{-3}) and binary cross-entropy and trained for 20 epochs with a batch size of 128 on the same 10,000 training images.

Layer	Output shape	Parameters
Conv2D(32,3)	$28 \times 28 \times 32$	320
MaxPooling2D(2)	$14 \times 14 \times 32$	0
Conv2D(64,3)	$14 \times 14 \times 64$	18,496
MaxPooling2D(2)	$7 \times 7 \times 64$	0
Conv2D(64,3) (latent feature map)	$7 \times 7 \times 64$	36,928
UpSampling2D(2)	$14 \times 14 \times 64$	0
Conv2D(32,3)	$14 \times 14 \times 32$	18,464
UpSampling2D(2)	$28 \times 28 \times 32$	0
Conv2D(1,3, sigmoid)	$28 \times 28 \times 1$	289
Total		74,497

The training and validation losses fell from 0.2228 and 0.1004 in the first epoch to 0.0683 and 0.0669 in the last, and the validation loss stayed at or below the training loss throughout, so no overfitting was observed. On the test set the CAE obtained an MSE of 0.002580, an MAE of 0.014898 and a mean SSIM of 0.974509.

The latent feature map of this architecture has $7 \times 7 \times 64 = 3136$ values, which is more than the 784 input pixels. The network therefore compresses the image spatially but not in the number of stored values, unlike the 16-value code of the FC-AE. This should be kept in mind when the two models are compared in Section 11, where a comparison with matched latent sizes is also reported.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022668	0.059465	0.728743	211,040
Convolutional AE	0.002580	0.014898	0.974509	74,497

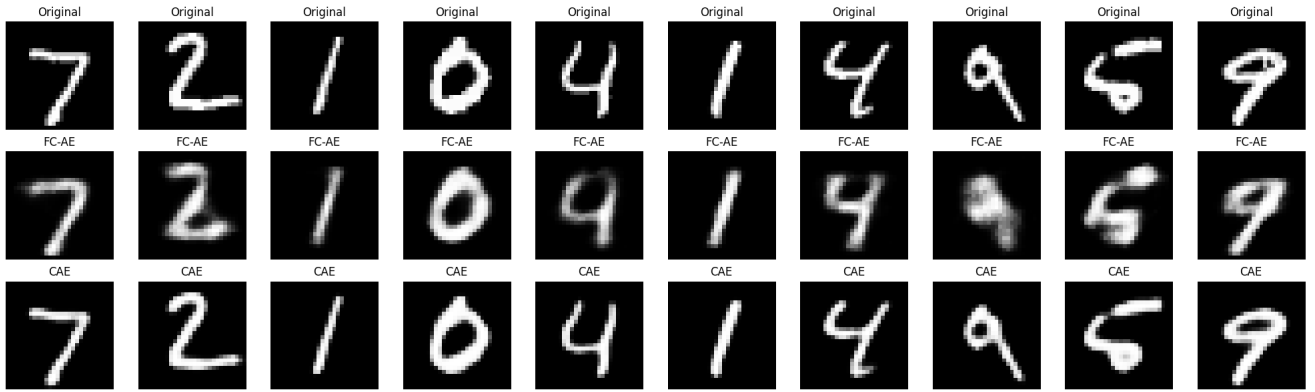
Plot 3: Reconstruction Comparison

Display:

Original $\rightarrow$ FC-AE Reconstruction $\rightarrow$ CAE Reconstruction.

Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.



Plot 3: Original images (top), FC-AE reconstructions (middle) and CAE reconstructions (bottom) for the same ten test images.

Comparison at matched latent size. Because the latent feature map of the CAE is much larger than the 16-value code of the FC-AE, a second comparison was made in which both architectures have a dense bottleneck of d values. The FC-AE results come from the latent-dimension study of Section 27 and the CAE results from Additional Exercise 1 (Section 29). Both used the same training protocol.

Latent dim d	FC-AE MSE	CAE MSE	FC-AE SSIM	CAE SSIM
8	0.026644	0.022872	0.687447	0.744092
16	0.022865	0.012497	0.730126	0.863173
32	0.020106	0.008689	0.758167	0.905526

Inference

What the plot shows. The same ten test digits reconstructed by the FC-AE and by the CAE, together with the metrics in the table above.

Trend observed. The CAE reconstructions are almost indistinguishable from the originals, whereas the FC-AE outputs are blurred and in a few cases change the apparent digit (the 4 and the 9). Numerically, the CAE has an MSE about 8.8 times lower (0.00258 against 0.02267), an MAE about 4 times lower and an SSIM of 0.975 against 0.729, while using about 2.8 times fewer parameters (74,497 against 211,040).

Why it may occur. Convolutional layers apply small shared filters to local neighbourhoods, so strokes, edges and curvature are captured with few parameters, and the decoder works on a 7×7 spatial map. The fully connected network flattens the image, loses the neighbourhood structure and must squeeze all information through 16 numbers. Part of the large gap is also due to the much larger latent map of the CAE.

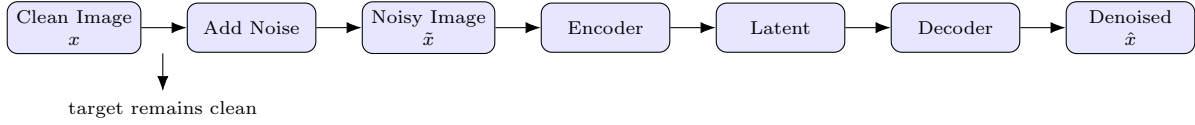
Conclusion. Preserving spatial locality through convolution improves reconstruction clearly. In the matched-size comparison the CAE is still better for every d (for example an MSE of 0.0125 against 0.0229 at $d = 16$, about 1.8 times lower, and an SSIM of 0.863 against 0.730), so convolution helps in itself, although the headline factor of 8.8 overstates its effect.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

Denoising Autoencoder: Implementation and Training

Noise generation. Two corruption types were implemented. Gaussian noise with standard deviation σ was added to the normalised images and the result was clipped to $[0, 1]$. For salt-and-pepper noise a proportion p of the pixels was replaced by extreme values, half of them set to 0 (pepper) and half to 1 (salt).

Model and training. The denoising autoencoder uses the same architecture as the CAE of Section 10 (74,497 parameters) with freshly initialised weights. It was trained for 20 epochs (Adam, learning rate 10^{-3} , batch size 128, binary cross-entropy) with Gaussian-noisy images ($\sigma = 0.2$) as input and the clean images as targets. The noisy training set was generated once and reused in every epoch. For validation, the test images were corrupted in the same way and compared with their clean versions. The training and validation losses decreased from 0.2676 and 0.1196 in the first epoch to 0.0746 and 0.0739 in the last.

Only Gaussian noise with $\sigma = 0.2$ was used for training. The other Gaussian levels and the salt-and-pepper noise were therefore applied to the trained model only at test time.

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

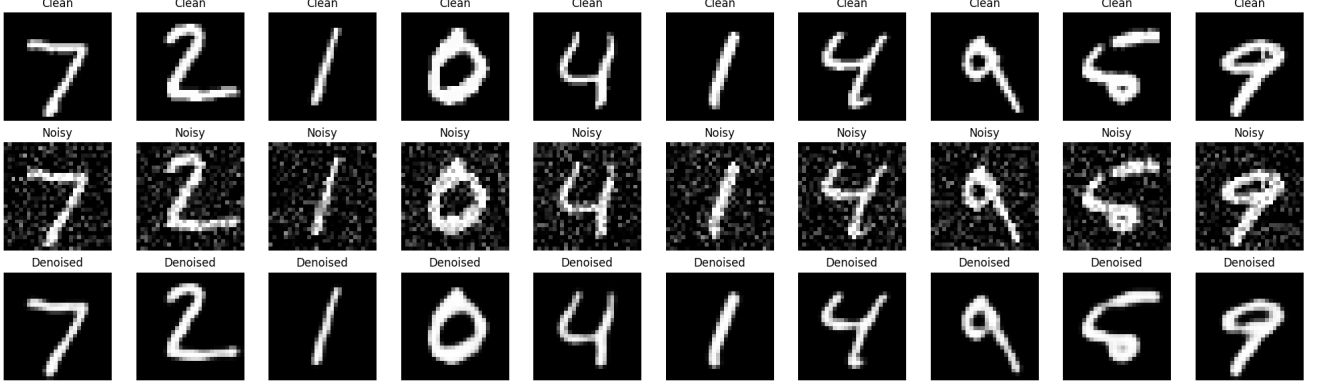
Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003409	0.017357	0.960372
0.2	0.004472	0.020807	0.946717
0.3	0.007147	0.028827	0.884323

Required inference:

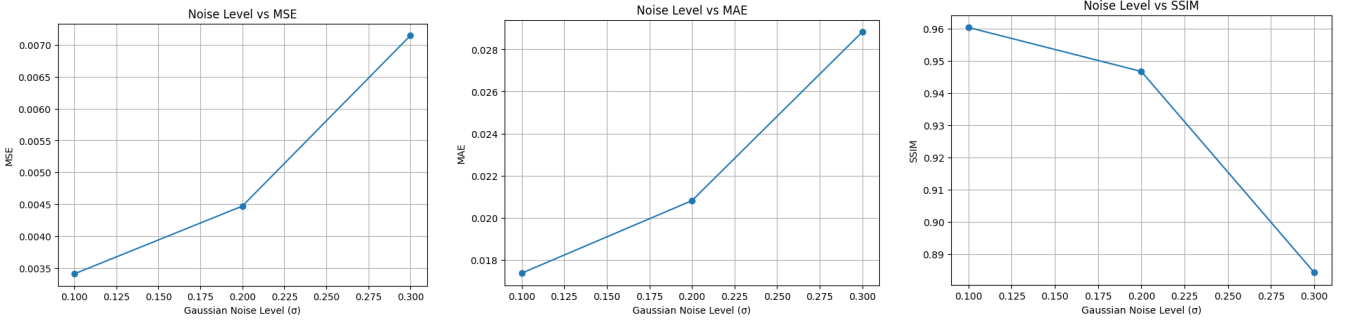
Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

Results: Plots 4 and 5



Plot 4: Clean test images (top), the same images with Gaussian noise of $\sigma = 0.2$ (middle) and the denoised reconstructions (bottom).

For the model evaluated at $\sigma = 0.2$ in the consolidated tables, the test metrics are $\text{MSE} = 0.004471$, $\text{MAE} = 0.020788$ and $\text{SSIM} = 0.946785$. These differ slightly from the $\sigma = 0.2$ row of the table above because the noise is drawn afresh in every evaluation.



Plot 5: Gaussian noise level σ against the MSE, MAE and SSIM of the denoised output. Each metric is shown on its own axis.

The table above is for Gaussian noise. For completeness, the same model was also evaluated on salt-and-pepper noise, which it had not seen during training.

Corruption level p	MSE	MAE	SSIM
0.05	0.004848	0.021361	0.928545
0.10	0.007533	0.028325	0.870612
0.20	0.015393	0.046202	0.721444

Inference

What the plot shows. The MSE, MAE and SSIM of the denoised output, measured against the clean images, for increasing noise levels, together with the denoised images of Plot 4.

Trend observed. For Gaussian noise the MSE increases from 0.00341 ($\sigma = 0.1$) to 0.00447 ($\sigma = 0.2$, an increase of 31%) and 0.00715 ($\sigma = 0.3$, a further increase of 60%). The MAE rises from 0.0174 to 0.0288 and the SSIM falls from 0.960 to 0.947 and then 0.884, so the degradation accelerates. Salt-and-pepper noise causes a faster decline (SSIM 0.929, 0.871 and 0.721 for $p = 0.05, 0.10$ and 0.20).

Why it may occur. Stronger noise destroys more of the stroke information, so the network must infer more from context. The model was trained at $\sigma = 0.2$, so $\sigma = 0.3$ lies outside its training conditions, which explains the steeper decline. Salt-and-pepper noise has a different character (isolated saturated pixels rather than small additive perturbations), so it also represents a mismatch with the training data. Even at $\sigma = 0.1$ the error (0.0034) is higher than for the CAE with clean input (0.0026), because information lost to the noise cannot be fully restored.

Conclusion. The denoising autoencoder recovers the major structure of the digits. In Plot 4 the background noise is removed, the strokes are continuous and every digit remains recognisable, with only slightly smoothed stroke thickness. Quality degrades gracefully up to $\sigma = 0.2$ (SSIM about 0.95) and more noticeably at $\sigma = 0.3$ and for heavy salt-and-pepper noise.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

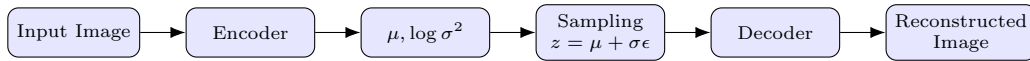
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

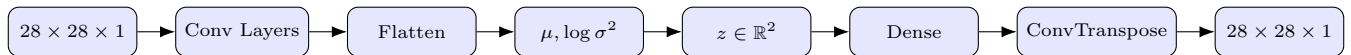
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



VAE Implementation and Training Results

The VAE was built as a single model with a two-dimensional latent space. The encoder (two convolution and pooling stages, a dense layer of 128 units) outputs μ and $\log \sigma^2$, each in $\mathbb{R}^2$. A latent vector is sampled with the reparameterization trick, $z = \mu + \sigma \odot \epsilon$, and the decoder maps it back to a $28 \times 28 \times 1$ image through a dense layer and two transposed convolutions. The reconstruction loss is the binary cross-entropy summed over the pixels of an image and averaged over the batch, and the KL term uses the closed form given in Section 16.

Part	Layers	Parameters
Encoder	Conv(32), MaxPool, Conv(64), MaxPool, Flatten (3136), Dense(128), Dense(4) for μ and $\log \sigma^2$	420,868
Decoder	Dense(3136), Reshape($7 \times 7 \times 64$), ConvT(64, stride 2), ConvT(32, stride 2), Conv(1, sigmoid)	65,089
Total		485,957

The model was trained with Adam (learning rate 10^{-3}), a batch size of 128 and 20 epochs. In the final epoch the training losses were 150.14 (reconstruction), 5.62 (KL) and 155.76 (total). The corresponding validation values, computed on the test images, were 151.29, 5.59 and 156.88.

18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

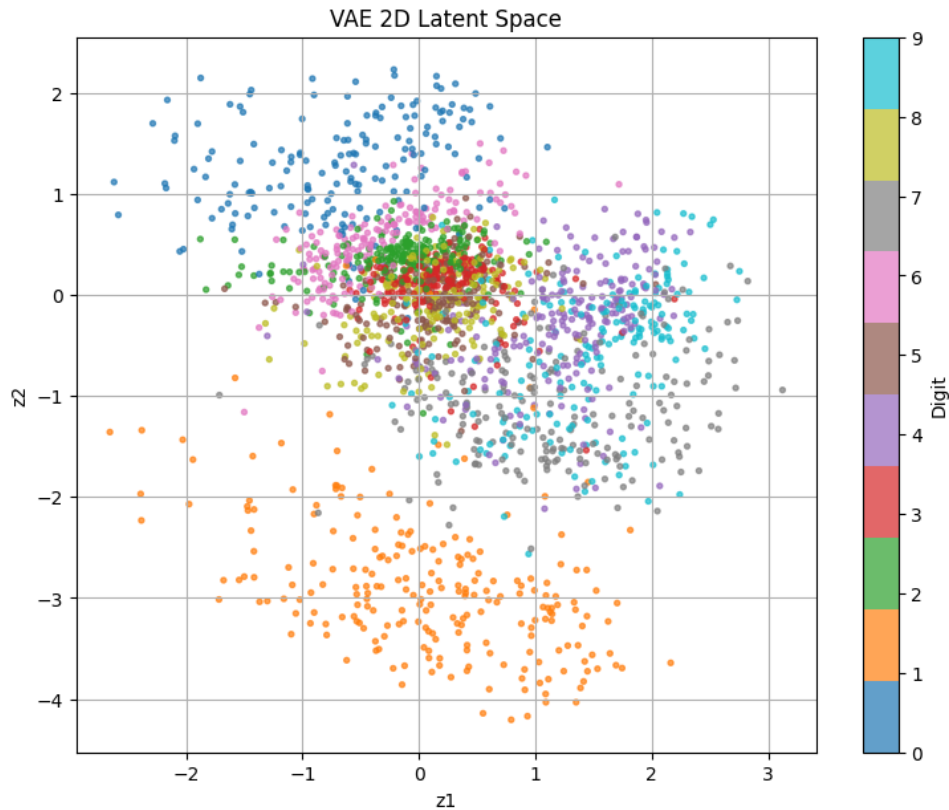
Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

Required inference:

Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.



Plot 6: Two-dimensional latent space of the VAE for the 2,000 test images (z_1 against z_2), coloured by digit label. The labels were used only for colouring.

Inference

What the plot shows. The sampled latent vector (z_1, z_2) of every test image, coloured by its true digit.

Trend observed. Digit 1 (orange) forms a well separated cluster at the bottom (z_2 between about -4 and -2) and digit 0 (blue) occupies the upper left. Digits 4 (purple), 9 (cyan) and 7 (grey) lie on the right and overlap strongly with one another, and digits 2, 3, 5, 6 and 8 are packed together close to the origin with heavy overlap. The whole cloud is centred near the origin, spans roughly -3 to 3 along z_1 and -4 to 2.5 along z_2 , and contains no large empty gaps.

Why it may occur. The KL term pulls every posterior towards $\mathcal{N}(0, I)$, which keeps the codes compact and centred, while the reconstruction term separates digits that look very different, such as 0 and 1. With only two dimensions, visually similar digits (4, 9 and 7, or 3, 5, 8, 2 and 6) cannot be separated. The labels were never given to the model, so this structure emerged from image similarity alone.

Conclusion. Samples of the same digit occupy similar regions, which shows that the latent space is meaningful, but different digits overlap considerably, with 0 and 1 as the main exceptions. A two-dimensional code is enough to reveal the structure but not to separate all ten classes.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

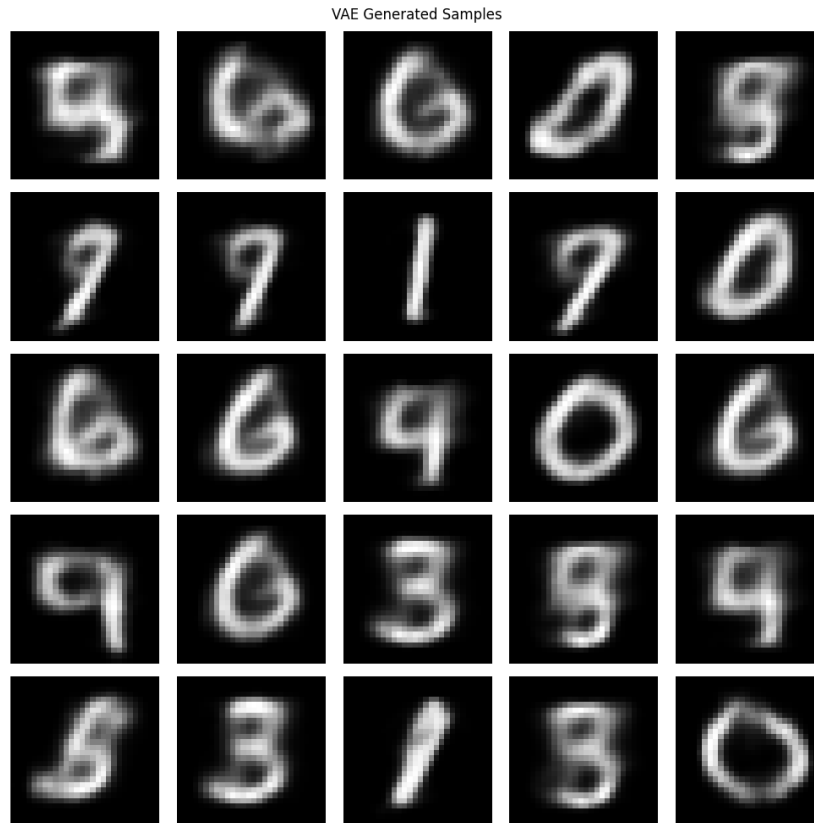
Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.



Plot 7: Twenty-five images generated by decoding random latent vectors $z \sim \mathcal{N}(0, I)$.

Assessment of the generated samples.

- **Resemblance to handwritten digits.** Most samples look like handwritten digits, but they are smoother and blurrier than real MNIST images.
- **Recognisable shapes.** Several samples are clearly recognisable, particularly 0, 1, 3, 5, 6 and 9. Many of the samples are 9-like or 7-like shapes.
- **Variation.** The samples differ in identity, slant, thickness and loop size, so the decoder does not produce a single prototype.
- **Ambiguous or unrealistic samples.** Some images look like a blend of two digits (for example 9 and 4, or 3 and 8) or have broken strokes, which is expected where the classes overlap in the latent space.

Inference

What the plot shows. A 5×5 grid of images decoded from random samples of the prior $\mathcal{N}(0, I)$.

Trend observed. Most images are digit-like and varied but blurry, and a minority are ambiguous blends. Some digits, particularly 2, appear rarely, and 4 is hard to tell apart from 9. The larger grid of 100 samples in Additional Exercise 7 shows the same behaviour.

Why it may occur. Samples drawn from the prior fall in regions the decoder has seen during training, because the KL term makes the distribution of the codes similar to $\mathcal{N}(0, I)$. The blur comes from the pixel-wise cross-entropy, which averages over plausible images, and from the small latent dimension, the short training (20 epochs) and the small training set. The blends appear where classes overlap in Plot 6.

Conclusion. The VAE works as a generative model, since new digit-like images can be produced from random noise, but the samples have limited sharpness and uneven coverage of the classes.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

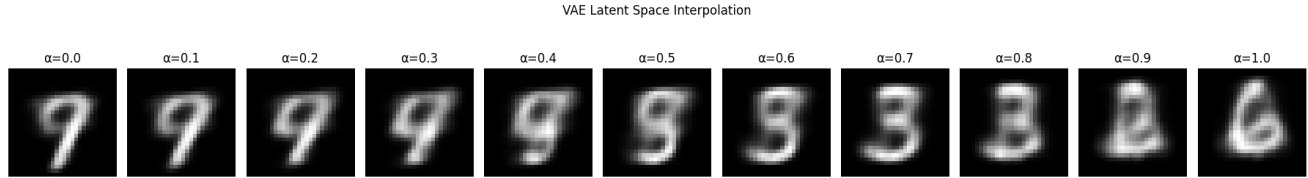
Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

Required inference:

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

Latent vectors used. The interpolation used the latent vectors of the first two test images (a 7 and a 2), $z_A = (0.889, -1.050)$ and $z_B = (-0.320, 0.581)$, with $\alpha = 0, 0.1, \dots, 1$.



Plot 8: Decoded images along $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$ for $\alpha = 0, 0.1, \dots, 1$.

Inference

What the plot shows. The decoder output for eleven equally spaced points on the straight line between two latent vectors.

Trend observed. The image changes gradually. It starts as a 9-like shape, which stays almost unchanged for the first four steps, then passes through 5-like and 3-like forms ($\alpha \approx 0.5-0.8$) and a 2- or 8-like shape at $\alpha = 0.9$, and ends as a 6-like shape at $\alpha = 1$. There are no abrupt jumps or noise-like images between neighbouring steps.

Why it may occur. The KL regularisation and the stochastic encoding force nearby latent points to decode to similar images, so the decoder behaves as a smooth function of z . The intermediate digits appear because the straight path crosses the crowded central region of Plot 6. The end points are decoded from sampled codes of a two-dimensional VAE, which reconstructs imperfectly, so the first and last images only approximate the original 7 and 2.

Conclusion. The latent space is continuous and smooth. Moving between two codes produces a gradual change in which every intermediate image is a plausible glyph rather than an artefact.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022668	0.059465	0.728743	211,040
Convolutional AE	0.002580	0.014898	0.974509	74,497
Denoising CAE ($\sigma = 0.2$)	0.004471	0.020788	0.946785	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	150.143799
KL Loss	5.617647
Total Loss	155.761490
Test MSE	0.044742
Test MAE	0.103395
Mean SSIM	0.467958

The Denoising CAE row was evaluated on test images corrupted with Gaussian noise ($\sigma = 0.2$) and compared with the clean images. The three VAE losses are the final-epoch training values. They are per-image sums of cross-entropy and KL divergence and are therefore not on the same scale as the MSE and MAE columns. The corresponding validation values were 151.29, 5.59 and 156.88.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.

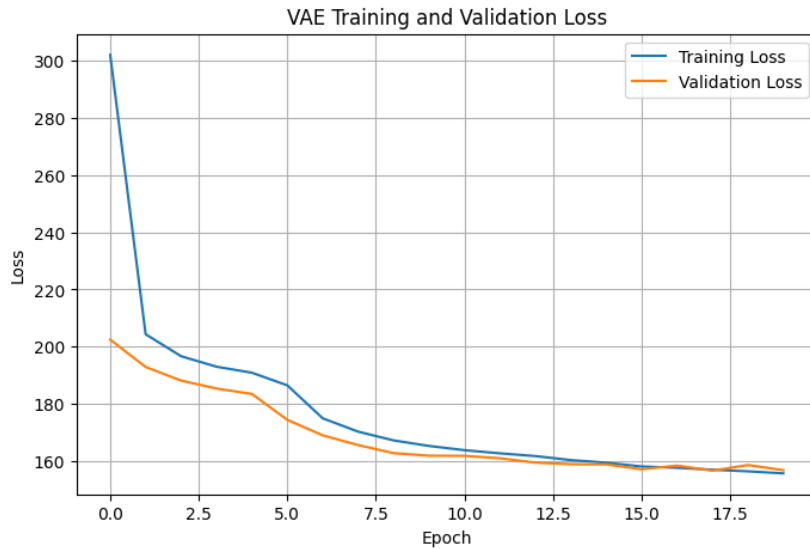
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

Plot Index and VAE Training Curve

The table below lists where each required plot appears. The numbering follows the section headings of this manual, and the VAE loss curve (item 9 of the list above) is given directly after the table.

No.	Required plot	Location
1	Original versus reconstructed image	Section 8 (Plot 1)
2	Training and validation loss, basic AE	Section 8 (Plot 2)
3	FC-AE versus CAE reconstruction	Section 11 (Plot 3)
4	Clean versus noisy versus denoised	Section 14 (Plot 4)
5	Noise level versus MSE, MAE and SSIM	Section 14 (Plot 5)
6	VAE latent space	Section 18 (Plot 6)
7	Randomly generated VAE images	Section 19 (Plot 7)
8	Latent-space interpolation	Section 20 (Plot 8)
9	Training and validation loss, VAE	Section 22 (below)
10	Reconstruction-error distribution	Section 23 (Plot 9)
–	Latent dimension versus MSE	Section 27 (Plot 10)



VAE training and validation total loss ($\mathcal{L}_{rec} + \mathcal{L}_{KL}$) for each epoch.

Inference

What the plot shows. The total VAE loss (summed cross-entropy plus KL divergence) on the training and validation sets for each epoch.

Trend observed. The training loss drops from 301.9 to 204.4 after the first epoch and reaches 155.8 at epoch 20. The validation loss goes from 202.5 to 156.9. Validation is lower than training in the early epochs, and from about epoch 15 the two curves almost coincide, with the validation loss fluctuating between about 156.7 and 158.9.

Why it may occur. The training loss is averaged over the epoch, which explains its high initial value. The validation loss is noisier because the random sample ϵ makes each evaluation stochastic. The gap between the curves stays within one to two units (about 1%).

Conclusion. The VAE trains stably and converges to a total loss of about 156–157 per image, with no sign of overfitting. The KL term stays small (about 5.6), so the two-dimensional posteriors remain close to the prior.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

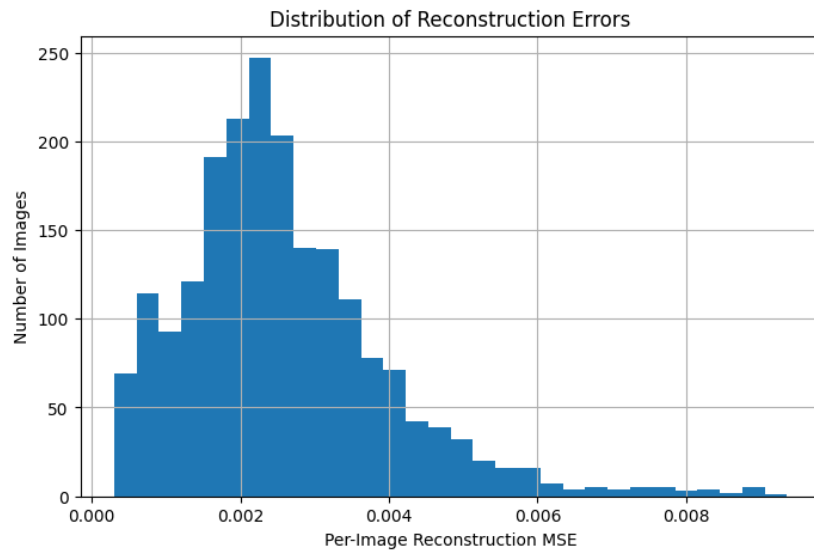
$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.



Plot 9: Histogram of the per-image reconstruction error e_i of the convolutional autoencoder over the 2,000 test images.

The per-image errors of the CAE range from 0.000302 to 0.009358, with a mean of 0.002580 (which equals the CAE test MSE).

- **Typical range.** Most images have an error between about 0.001 and 0.004, and the histogram peaks near 0.002 to 0.0025.
- **Concentration.** The distribution is concentrated and right-skewed, with a compact main body and a thin tail reaching 0.009.
- **High-error samples.** A few dozen images (roughly 2% of the test set) have an error above 0.006, which is more than twice the mean.
- **Representative images.** The five images with the highest error are shown and discussed in Section 24.

Inference

What the plot shows. The distribution of the per-image MSE of the CAE over the test set.

Trend observed. A single-peaked, right-skewed histogram with its mode near 0.0023, most of its mass between 0.0005 and 0.005, and a long thin tail up to 0.0094.

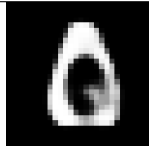
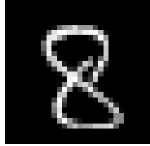
Why it may occur. Most digits consist of ordinary strokes that the CAE reproduces almost perfectly, whereas a few unusual or complex glyphs (Section 24) cannot be represented as accurately.

Conclusion. Reconstruction error is low for the typical image and high only for a small number of outliers. The error agrees with visual quality: images near the mode are almost exact copies, while the tail contains the visibly imperfect ones. The histogram could therefore also be used to flag unusual images.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	Original	Reconstruction
1 (test index 895, $e = 0.009358$)		
2 (test index 1982, $e = 0.009021$)		
3 (test index 431, $e = 0.008920$)		
4 (test index 1395, $e = 0.008903$)		
5 (test index 1124, $e = 0.008836$)		

Students should provide a short explanation for why these images may be difficult to reconstruct.

The five worst reconstructions of the CAE have errors between 0.0088 and 0.0094, which is about 3.4 to 3.6 times the average. All five are atypical handwriting. Sample 1 is an unusually wide and heavily filled glyph, and the network leaves an uneven, blurred interior. Samples 2 and 4 contain long or overshooting strokes and several overlapping loops, which are rare in the training data. Sample 3 has very thin strokes that cross each other, and thin lines are hard to reproduce exactly because their edges tend to become thicker or fainter. Sample 5 is thin and fragmented, so parts of its strokes are smeared or dropped. In every case the overall digit is still reproduced, and the error comes mainly from stroke thickness, intensity and fine detail rather than from a wrong digit. Such digits are probably rare among the 10,000 training images, and a pixel-wise loss penalises small misalignments of thin strokes heavily.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.022668	0.059465	0.728743	211,040	≈ 9 s
Conv. Autoencoder	0.002580	0.014898	0.974509	74,497	≈ 31 s
Denoising CAE	0.004471	0.020788	0.946785	74,497	≈ 26 s
VAE	0.044742	0.103395	0.467958	485,957	≈ 28 s

The values must be obtained from the student's own execution.

All metrics were computed on the 2,000 test images. For the Denoising CAE they compare the denoised output of Gaussian-corrupted images ($\sigma = 0.2$) with the clean images, and for the VAE they are based on its reconstructions of clean test images. The time is the total training time for 20 epochs, obtained by adding up the per-epoch times printed by Keras. It is therefore approximate, since the per-epoch times are rounded to whole seconds and the first epoch includes graph compilation.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

Consolidated Inferences

The ten mandatory inferences are summarised below. The remaining plot-specific inferences are given next to the corresponding plots.

1. **Effect of latent dimension on reconstruction.** For the FC-AE, the test MSE decreases from 0.0552 to 0.0266, 0.0229 and 0.0201 as d_z increases from 2 to 8, 16 and 32, while the SSIM rises from 0.318 to 0.687, 0.730 and 0.758. The returns diminish quickly: going from 2 to 8 halves the MSE, whereas the steps from 8 to 16 and from 16 to 32 reduce it by only about 12–14% each. A larger code stores more information, but MNIST digits have a limited number of independent factors of variation, so the additional dimensions add little. There is a trade-off between compression and fidelity, and a moderate code of 8–16 values gives most of the achievable quality.
2. **Fully connected versus convolutional reconstruction.** The CAE is far better (MSE 0.00258 against 0.02267, SSIM 0.975 against 0.729) with fewer parameters, because shared local filters exploit the two-dimensional structure of the image. Part of this gap comes from the larger latent map of the CAE, but with equal latent size the CAE is still 1.2 to 2.3 times better in MSE (factors of 1.2, 1.8 and 2.3 at $d = 8, 16$ and 32).
3. **Effect of Gaussian noise level.** For $\sigma = 0.1, 0.2$ and 0.3 the MSE increases from 0.0034 to 0.0045 and 0.0071 and the SSIM falls from 0.960 to 0.947 and 0.884. More noise removes more information, and $\sigma = 0.3$ also exceeds the noise level used in training.
4. **Ability of the denoising autoencoder to recover structure.** At $\sigma = 0.2$ the denoising CAE removes the background noise and restores continuous, recognisable strokes (SSIM 0.947). It therefore recovers the major structure of the digits, and only fine stroke-thickness detail is lost.
5. **Characteristics of the VAE latent distribution.** The encoded test images form one compact cloud centred near the origin, extending about three units along each axis, without large gaps. This is consistent with the KL pull towards $\mathcal{N}(0, I)$ (final KL about 5.6), although the cloud is not an exact standard normal.
6. **Structure of the two-dimensional latent space.** Digits 1 and 0 occupy separate regions (bottom and upper left), digits 4, 9 and 7 overlap on the right, and digits 2, 3, 5, 6 and 8 overlap in the centre. Images of the same digit are neighbours, but many classes cannot be separated in two dimensions.
7. **Quality and diversity of generated samples.** The generated digits are varied and mostly digit-like but blurry, with some ambiguous blends, and some classes (for example 2) occur rarely.
8. **Smoothness of latent-space interpolation.** The decoded images change gradually along both interpolation paths (the pair of the first two test images and the digit 0 to digit 1 path) without abrupt jumps. The intermediate images resemble plausible digits of other classes (for example 5, 3 and 6).
9. **Relationship between reconstruction error and visual quality.** Images with a low error (near 0.002) are almost exact copies, while the highest errors (about 0.009) belong to thick, thin or calligraphic glyphs whose strokes are visibly smeared. The MSE ranks the models consistently with the SSIM and with visual impression (CAE, then denoising CAE, then FC-AE, then VAE), although the SSIM shows blur more clearly than the MSE.
10. **Deterministic versus probabilistic latent representations.** The fully connected and convolutional autoencoders map each image to a single point in an unconstrained latent space, which gives sharper reconstructions but no guarantee that random or interpolated points decode to valid digits. The VAE encodes an image as a distribution regularised towards a common prior, which gives a continuous space that can be sampled (Plots 6 to 8) at the price of an additional KL term, blurrier images and, for the two-dimensional model, lower fidelity (MSE 0.0447, SSIM 0.468). Doubling the KL weight ($\beta = 2$) lowered the KL term from 5.62 to 4.77 and slightly worsened the reconstruction (Additional Exercise 6).

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

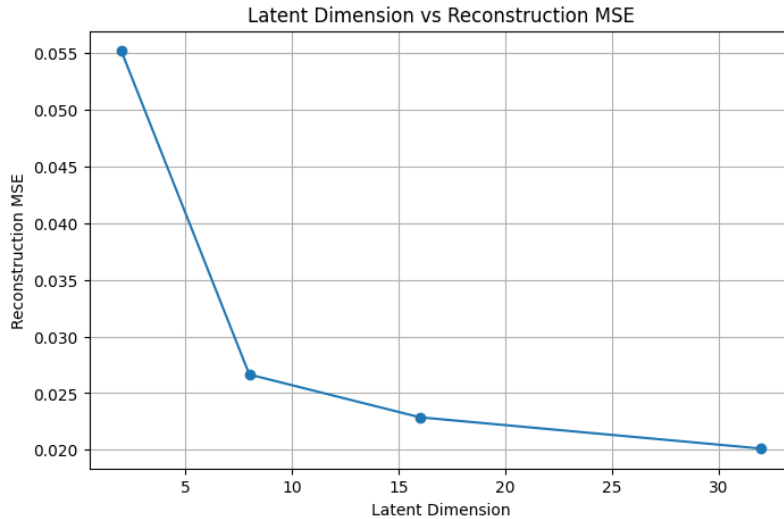
Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.055202	0.317665	210,130
8	0.026644	0.687447	210,520
16	0.022865	0.730126	211,040
32	0.020106	0.758167	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE

Students should explain the trade-off between compression and reconstruction quality.

The same architecture and training protocol as in Section 7 were used, and only the size of the latent layer was changed. The value for $d_z = 16$ differs slightly from the main run of Section 9 (0.022668) because of the random weight initialisation.



Plot 10: Latent dimension against the test reconstruction MSE of the FC-AE.

Inference

What the plot shows. The test MSE of the FC-AE for latent dimensions of 2, 8, 16 and 32.

Trend observed. The MSE drops sharply from 0.0552 ($d_z = 2$) to 0.0266 ($d_z = 8$) and then only slowly to 0.0229 ($d_z = 16$) and 0.0201 ($d_z = 32$). The SSIM increases correspondingly from 0.318 to 0.758.

Why it may occur. A code of two values cannot hold the style, slant and shape of a digit, so different digits are blurred together. Beyond about 8 to 16 dimensions most of the important variability is already stored, and further dimensions bring diminishing gains.

Conclusion. There is a trade-off between compression and reconstruction quality. Smaller codes compress more but lose detail, and larger codes reconstruct better but compress less (even 32 values is still a 24.5-fold reduction from 784). A dimension of 16 is a good compromise here.

28. Discussion Questions

1. What is an autoencoder?

Answer. An autoencoder is a neural network trained to reproduce its input at its output through a lower-dimensional bottleneck. It consists of an encoder $z = f_\theta(x)$ and a decoder $\hat{x} = g_\phi(z)$ and needs no labels, because the input itself is the target.

2. What is the purpose of the encoder?

Answer. The encoder compresses the input into a compact latent code that keeps the information needed for reconstruction and discards redundancy.

3. What is the purpose of the decoder?

Answer. The decoder reconstructs the input from the latent code by mapping it back to image space. A sigmoid output keeps the pixel values in $[0, 1]$.

4. What is a latent representation?

Answer. It is the compressed feature vector (or feature map) at the bottleneck of the network, for example the 16-value code of the FC-AE or the two values (z_1, z_2) of the VAE.

5. Why is the latent representation usually smaller than the input?

Answer. A bottleneck forces the network to learn the essential structure of the data instead of copying it, since an identity mapping would learn nothing useful. In this experiment the CAE is an exception, because its $7 \times 7 \times 64$ latent map is larger than the input.

6. What happens when the latent dimension is too small?
Answer. Too much information is lost, so the reconstructions become blurred or change identity. With $d_z = 2$ the FC-AE reached an MSE of 0.0552 and an SSIM of only 0.318.
7. What happens when the latent dimension is increased?
Answer. The reconstruction improves with diminishing returns (from $d_z = 2$ to 32 the MSE fell from 0.0552 to 0.0201 and the SSIM rose from 0.318 to 0.758), but the compression decreases, and a very large code approaches an identity mapping.
8. Why can a convolutional autoencoder preserve spatial information more effectively?
Answer. Convolutions use local receptive fields and shared weights, so neighbouring pixels are processed together, strokes are detected wherever they occur, and a spatial feature map is kept instead of a flattened vector. Here the CAE reached an SSIM of 0.975 against 0.729 for the FC-AE with about 2.8 times fewer parameters.
9. Differentiate an autoencoder and a denoising autoencoder.
Answer. A standard autoencoder receives a clean image and reproduces it. A denoising autoencoder receives a corrupted image $\tilde{x}$ and must reproduce the clean image x , so it has to learn the structure of the data instead of simply copying its input.
10. Why is the clean image used as the target in denoising?
Answer. The aim is to remove the corruption. If the noisy image were the target, the network would learn to reproduce the noise, whereas a clean target makes the loss measure directly how well the noise has been removed.
11. What happens when the noise level is increased?
Answer. More information is destroyed and the reconstruction quality drops. For $\sigma = 0.1, 0.2$ and 0.3 the MSE was 0.0034, 0.0045 and 0.0071 and the SSIM was 0.960, 0.947 and 0.884.
12. Why are MSE, MAE and SSIM useful for image reconstruction?
Answer. The MSE penalises large pixel errors, the MAE gives the average error in intensity units and is less affected by outliers, and the SSIM compares local luminance, contrast and structure, which is closer to visual similarity. Together they describe both pixel accuracy and structural fidelity. The MSE alone can look small for blurred MNIST images because most pixels are black background.
13. What is a Variational Autoencoder?
Answer. A VAE is a generative autoencoder whose encoder outputs a distribution $q_\phi(z|x) = \mathcal{N}(\mu, \sigma^2)$ over the latent code, and which is trained by minimising a reconstruction loss plus the KL divergence to a prior.
14. How does a VAE differ from a conventional autoencoder?
Answer. A VAE encodes an image as a distribution instead of a single point, samples z during training, and adds a KL term that pulls the codes towards $\mathcal{N}(0, I)$. It can therefore generate new images by sampling from the prior.
15. Why does a VAE learn a distribution instead of a single latent vector?
Answer. Learning a distribution that is regularised towards a common prior makes the latent space continuous and well filled, so that nearby points decode to similar images and random samples decode to plausible images.
16. What is the reparameterization trick?
Answer. It consists of writing $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. The randomness is moved into ϵ , so gradients can flow through μ and σ to the encoder.
17. Why is ϵ sampled from a standard normal distribution?
Answer. It is a fixed noise source without parameters, with zero mean and unit variance, which μ and σ then shift and scale. It matches the standard normal prior and keeps the KL term in closed form.
18. What is the role of KL divergence in the VAE loss?
Answer. It regularises the posterior towards the prior, which keeps the latent space compact, centred and suitable for sampling. It competes with the reconstruction term: with $\beta = 2$ the KL fell from 5.62 to 4.77 while the MSE rose slightly from 0.0447 to 0.0457.
19. What is the prior distribution used in the VAE?
Answer. The prior is $p(z) = \mathcal{N}(0, I)$, here a two-dimensional standard normal distribution.
20. Why is a two-dimensional latent space useful for visualization?
Answer. It can be plotted directly, so the clustering and overlap of the digits can be examined without any dimensionality reduction. The price is a limited capacity (SSIM 0.468).
21. What does latent-space interpolation demonstrate?
Answer. It shows that the latent space is continuous and organised, since moving between two codes produces a gradual change in the decoded image (Plot 8).
22. Why can a VAE generate new images?
Answer. Its codes are trained to follow the prior approximately, so points sampled from $\mathcal{N}(0, I)$ fall in regions that the decoder has learned to map to images.
23. Why might some VAE-generated images be unclear?
Answer. The two-dimensional latent space is small, so classes overlap and samples from overlapping or low-density regions decode to blends. In addition, the pixel-wise cross-entropy averages over plausible images, which causes blur, and the training was short (20 epochs on 10,000 images).
24. What is the difference between reconstruction and generation?

Answer. Reconstruction encodes a real image and decodes it, which measures fidelity. Generation decodes a code sampled from the prior without any input image, which measures the quality of the learned latent distribution.

25. Why should test images remain unseen during training?

Answer. Unseen test images give an unbiased estimate of generalisation, whereas seen images would measure memorisation. In this notebook the test images were used only as validation data to monitor the loss and never to update the weights or select a model, although a separate validation split would be the cleaner protocol.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.

Result. For each d , a CAE with a true dense latent bottleneck was trained (flatten, dense layer of size d , dense layer of 3136 units, reshape to $7 \times 7 \times 64$, then up-sampling and convolution).

Latent dim	MSE	SSIM
4	0.035461	0.610713
8	0.022872	0.744092
16	0.012497	0.863173
32	0.008689	0.905526

The quality improves steadily with d (the MSE falls by a factor of 4.1 from $d = 4$ to $d = 32$) with diminishing returns. All values are clearly worse than for the main CAE (MSE 0.00258), whose latent map has 3136 values, which confirms that the size of the bottleneck largely determines the reconstruction quality.

2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.

Result. The denoising CAE trained with Gaussian noise ($\sigma = 0.2$) was evaluated on Gaussian noise with $\sigma = 0.2$ and on salt-and-pepper noise with $p = 0.10$.

Noise type	MSE	MAE	SSIM
Gaussian ($\sigma = 0.2$)	0.004495	0.020874	0.946180
Salt-and-pepper ($p = 0.10$)	0.007520	0.028352	0.871019

The error is about 1.7 times higher for salt-and-pepper noise (SSIM 0.871 against 0.946). A likely reason is that the model was trained on Gaussian noise only, and saturated black or white outlier pixels are harder to distinguish from genuine strokes. The comparison is therefore not entirely fair to the salt-and-pepper case.

3. Train the denoising autoencoder using two different noise levels and compare SSIM.

Result. The model trained at $\sigma = 0.2$ was evaluated at two other noise levels. At $\sigma = 0.1$ the SSIM was 0.960584 and at $\sigma = 0.3$ it was 0.882865, compared with 0.946180 at $\sigma = 0.2$. Lower noise therefore gives a higher SSIM. A separate model was not retrained for each training noise level, so this comparison concerns the noise level at test time.

4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.

Result. A CAE whose decoder uses two transposed-convolution layers (stride 2, with 32 and 16 filters) instead of up-sampling followed by convolution (78,977 parameters) reached an MSE of 0.002287, an MAE of 0.013980 and an SSIM of 0.976849. The up-sampling version (74,497 parameters) gave 0.002580, 0.014898 and 0.974509. The transposed-convolution decoder is slightly better (about 11% lower MSE), probably because the up-sampling is learned instead of fixed. The difference is small and comes from single runs.

5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.

Result.

Metric	VAE, $d = 2$	VAE, $d = 8$
Reconstruction loss (final epoch)	150.14	93.53
KL loss (final epoch)	5.62	16.66
Total loss (final epoch)	155.76	110.19
Test MSE	0.044742	0.020819
Test MAE	0.103395	0.054279
Mean SSIM	0.467958	0.774152

The eight-dimensional VAE reconstructs much better (MSE 2.1 times lower) and stores more information in its code (KL of 16.7 against 5.6). The two-dimensional VAE has the smaller KL term and can be plotted directly (Plot 6), but its classes overlap. The eight-dimensional latent space cannot be visualised directly.

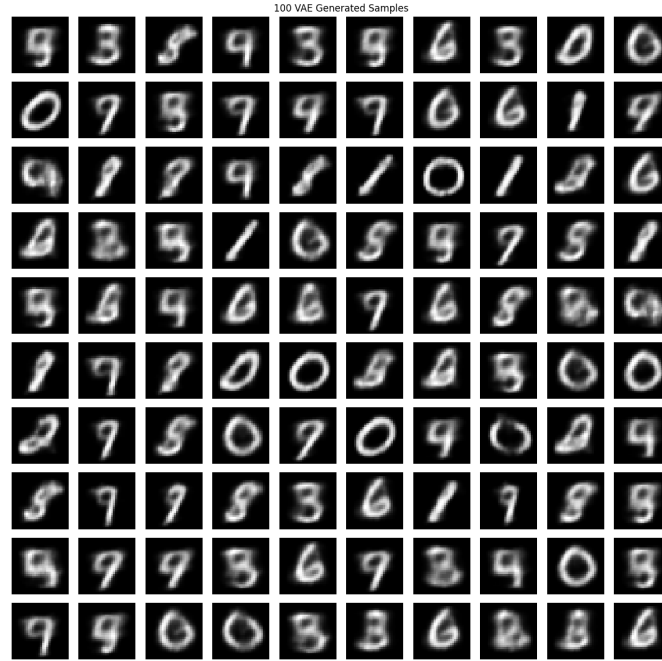
6. Investigate the effect of increasing the KL-loss contribution.

Result. A β -VAE with the objective $\mathcal{L} = \mathcal{L}_{rec} + \beta \mathcal{L}_{KL}$ and $\beta = 2$ was trained with a two-dimensional latent space and compared with the standard VAE ($\beta = 1$).

Metric	$\beta = 1$	$\beta = 2$
Reconstruction loss	150.14	151.03
KL loss	5.62	4.77
Test MSE	0.044742	0.045736
Test MAE	0.103395	0.105790
Mean SSIM	0.467958	0.463218

Doubling the KL weight lowered the KL term by about 15% and slightly worsened the reconstruction, as expected from the stronger pull towards the prior. The effect is small for this range of β and is based on a single run for each setting. The total losses are not compared, because the two objectives differ.

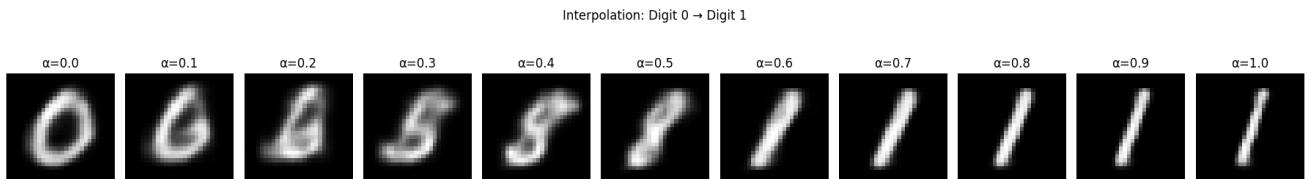
7. Generate a larger grid of VAE samples and discuss diversity.



One hundred images generated from $z \sim \mathcal{N}(0, I)$ with the two-dimensional VAE.

Discussion. The 100 samples contain shapes resembling 0, 1, 3, 5, 6, 7 and 9 with different slants, widths and stroke thicknesses, so there is real diversity. A considerable share of the samples are 9-like or 7-like, digits such as 2 appear rarely, and a number of samples are blurred blends (3, 8 or 5-like shapes, or broken loops) that cannot be assigned clearly to any digit.

8. Select two digit regions in the latent space and perform interpolation.



Interpolation between the latent codes of a test digit 0 ($z_A = (-0.956, 1.049)$) and a test digit 1 ($z_B = (-0.899, -3.144)$).

Discussion. The image changes smoothly from a 0 at $\alpha = 0$ through 6-like shapes ($\alpha = 0.1-0.2$) and 5- or 3-like shapes ($\alpha = 0.3-0.5$) to a slanted 1 from about $\alpha = 0.6$ onwards, which becomes thinner as α approaches 1. This agrees with Plot 6, where the straight path leaves the region of the 0 at the upper left, crosses the central area occupied by the digits 2, 3, 5, 6 and 8, and enters the isolated region of the 1 at the bottom.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

Conclusion

The complete pipeline was executed on 10,000 MNIST training images and 2,000 test images. The convolutional autoencoder gave the best reconstructions (SSIM 0.975), while the fully connected autoencoder with a 16-value code was considerably blurrier (SSIM 0.729). The denoising autoencoder removed Gaussian noise well (SSIM 0.947 at $\sigma = 0.2$) and degraded gracefully as the noise increased. The two-dimensional VAE learned a structured and smooth latent space from which new digits could be generated, at the cost of a lower reconstruction fidelity (SSIM 0.468).

Some limitations should be kept in mind when interpreting the numbers. All values come from single training runs without a fixed random seed, so small differences (for example between the two decoders in Exercise 4) lie within run-to-run variation. The latent feature map of the main CAE is larger than the input, so part of its advantage over the FC-AE is explained by the absence of a compressive bottleneck, which is why the matched-size comparison of Section 11 is included. The test images served only as validation data, and the VAE losses in the tables are final-epoch training values. Finally, the denoising model was trained on Gaussian noise with $\sigma = 0.2$ only, so the results for other noise levels and for salt-and-pepper noise show how well it generalises at test time.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>
8. Source code of this experiment: https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_7_autoencoders